hashlock.

2024

Security Audit

RDDTOR (dApp)

August 2024 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

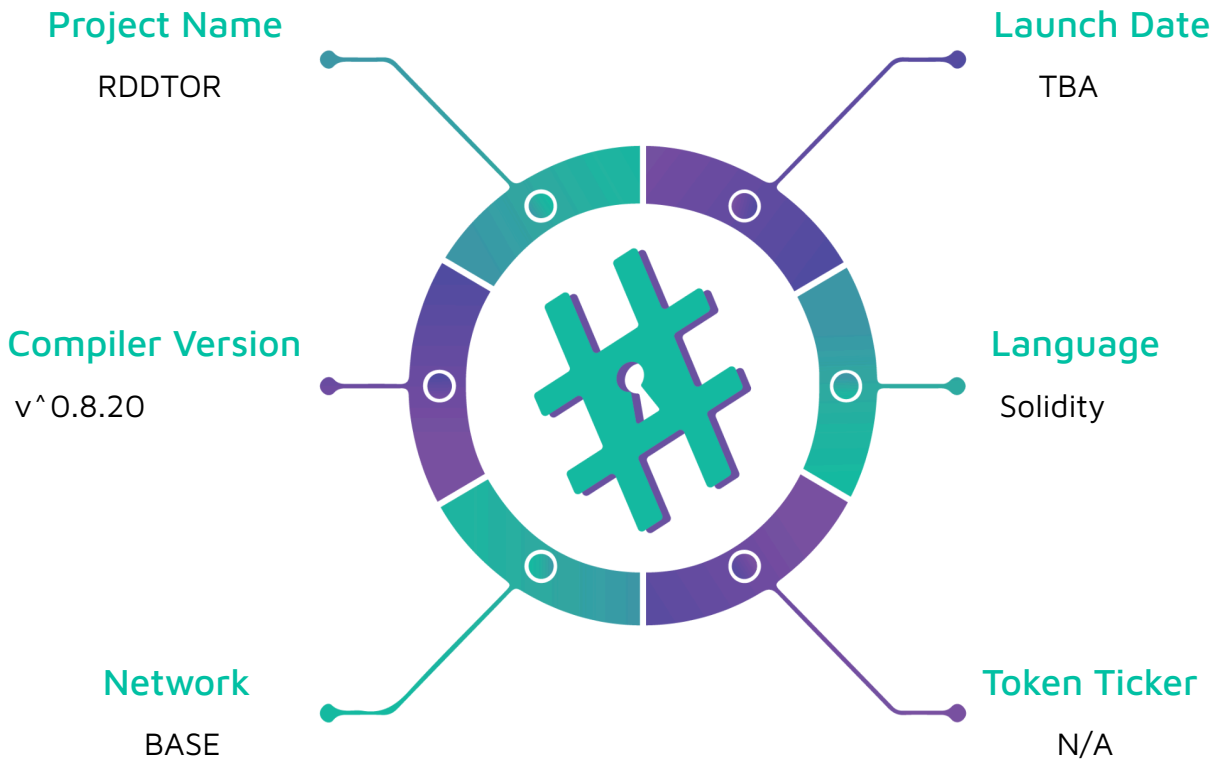
The RDDT team partnered with Hashlock to conduct a security audit of their ProfileWorkflow smart contract. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed contracts were secure.

Project Context

RDDTOR is a decentralized social platform that integrates cryptocurrency elements to enhance user experiences in social media and finance. It utilizes its native $RDDTOR token (on Base Ethereum Layer 2) to facilitate various transactions and interactions within its ecosystem. The platform also leverages the $RDDT token (an ERC-20 token on Ethereum), which is essential for governance voting, staking rewards, and receiving a share of the platform's revenue. RDDTOR aims to provide a fair and transparent environment through consensus-based and AI-driven content moderation, while offering revenue-sharing opportunities specifically for content creators and $RDDT token holders.

Project Name: RDDTOR
Compiler Version: ^0.8.20
Website: https://rddt.io/
Logo:

Visualised Context:



Project Visuals:

Audit scope

We at Hashlock audited the solidity code within the RDDTOR project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**RDDTOR Protocol Smart Contracts**
Platform	**BASE/ Solidity**
Audit Date	**August, 2024**
Contract 1	Contract.sol
Contract 1 MD5 Hash	a584f427078905815cd3333d3489d2ae
Contract 2	WorkflowBaseUpgradeable.sol
Contract 2 MD5 Hash	cd18bc83b4a0c44e187ebb382d11e470
Contract 3	WorkflowBaseCommon.sol
Contract 3 MD5 Hash	f4b607ae39577d75ce0b19cab2e98c5e
Contract 4	NonTransferrableERC721Upgradeable.sol
Contract 4 MD5 Hash	187b051bfae27fcdea10fa3d42c7790e
GitHub Commit Hash	c7bda26c67f89df274d2eddbdedf01384f61b81f

Security Rating

After Hashlock's Audit, we found the smart contracts to be "Secure". The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section.

We initially identified some significant vulnerabilities that have since been addressed.

Hashlock found:

5 Medium-severity vulnerabilities

4 Low-severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour

Contract.sol
- Profile Management:
  - Allows creation and management of decentralized social network profiles as non-transferrable, soulbound NFTs.
- Profile Registration:
  - Users can register a profile by providing a name, which initializes the profile in an "Active" state and assigns the caller's address as the wallet owner.
- Profile Updates:
  - Enables changing the name of an existing profile, ensuring the caller is the profile's wallet owner and maintaining the profile's "Active" status.
- Ownership and Access Control:
  - Utilizes an ownership model to restrict access to profile modifications, ensuring only the profile owner can make changes.
- Data Retrieval:
  - Provides functions to retrieve profile details, such as ID, status, wallet address, image, name, and description, based on profile ID or wallet address.
	Contract achieves this functionality.

WorkflowBaseUpgradeable.sol - Provides an initialization mechanism for upgradeable contracts, ensuring that base initialization logic is executed properly.	**Contract achieves this functionality.**

WorkflowBaseCommon.sol - Manages a counter to generate unique IDs and provides functions to retrieve the latest IDs or a specific range of IDs. - It emits events when items are updated, allowing changes to be tracked. - Supports adding and removing items from a mapping that associates foreign keys with arrays of item IDs. - Converts uint256 values to their string representation. - Implements safe transfer functions for transferring tokens between addresses, ensuring successful execution or reverting otherwise. - Interfaces with an external service locator to retrieve service addresses.	**Contract achieves this functionality.**
NonTransferrableERC721Upgradeable.sol - Provides initialization logic for upgradeable ERC721 contracts. - Overrides the update mechanism to enforce non-transferability of tokens, ensuring tokens cannot be transferred once minted.	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts for the RDDTOR project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the RDDTOR project smart contract code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help us understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

Medium

[M-01] ProfileWorkflow - Incorrect Imports Causing Compilation Errors

Description

The smart contract is experiencing compilation errors due to incorrect import statements. The errors arise from multiple declarations of the same identifiers, leading to conflicts during compilation.

Vulnerability Details

The errors are caused by importing contracts from different sources that declare the same identifiers. Specifically, the `Initializable` and `ERC721Upgradeable` contracts are imported multiple times from different paths, resulting in `DeclarationError: Identifier already declared`. This prevents the contract from compiling successfully.

Affected Imports

- `NonTransferrableERC721Upgradeable` and `WorkflowBaseUpgradeable` from `Ideevoog/RDDTOR.Token` repository.
- `Initializable` and `ERC721Upgradeable` from OpenZeppelin's upgradeable contracts.

Recommendation

To resolve these compilation errors, follow these steps:

1. Consolidate Imports:
- Use a single source for OpenZeppelin contracts to avoid conflicts. Prefer using the local package manager (e.g., npm) for consistent versioning.
- Example:

```
import "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";
```

```
import "@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol";

import "@openzeppelin/contracts-upgradeable/token/ERC721/ERC721Upgradeable.sol";
```

2. Use Local Copies for Custom Contracts:

- For custom contracts like NonTransferrableERC721Upgradeable and WorkflowBaseUpgradeable, ensure they are included in your project directory and import them using relative paths.
- Example:

```
import "./NonTransferrableERC721Upgradeable.sol";

import "./WorkflowBaseUpgradeable.sol";
```

3. Remove Redundant Imports:

- Ensure that each contract is only imported once and from a single source to prevent identifier clashes.

Status

Resolved

[M-02] ProfileWorkflow - Missing Name Validation

Description

The `ProfileWorkflow` contract lacks proper validation for the `name` parameter when registering or changing a profile. This can lead to the acceptance of invalid or undesirable names, potentially causing issues with profile management and user experience.

Vulnerability Details

The contract currently allows any string to be used as a profile name without validation. This can lead to several issues:

- Acceptance of empty or excessively long names.
- Names with leading or trailing spaces.
- Names with continuous spaces or special characters that are not allowed.

```
function registerProfile(string calldata name) public returns (uint256) {

    // Name validation is missing here

    ...

}


function changeName(uint256 id, string calldata name) public returns (uint256) {

    // Name validation is missing here

    ...

}
```

Recommendation

- Integrate the `validateName` function into the `registerProfile` and `changeName` functions to ensure that only valid names are accepted. This will prevent invalid data from being stored and improve the robustness of the contract.

```
    function validateName(string memory str) public pure returns (bool) {

        bytes memory b = bytes(str);

        if (b.length < 1) return false;

        if (b.length > 25) return false; // Cannot be longer than 25 characters

        if (b[0] == 0x20) return false; // Leading space

        if (b[b.length - 1] == 0x20) return false; // Trailing space


        bytes1 lastChar = b[0];


        for (uint256 i; i < b.length; i++) {

            bytes1 char = b[i];
```

```
                if (char == 0x20 && lastChar == 0x20) return false; // Cannot contain
continous spaces


            if (
                !(char >= 0x30 && char <= 0x39) && //9-0
                !(char >= 0x41 && char <= 0x5A) && //A-Z
                !(char >= 0x61 && char <= 0x7A) && //a-z
                !(char == 0x20) //space
            ) return false;


            lastChar = char;
        }


        return true;
    }
```

Status

Resolved

[M-03] ProfileWorkflow#changeName - Lack of Name Reservation Mechanism

Description

The `changeName` function in the `ProfileWorkflow` contract lacks a mechanism to reserve or de-reserve profile names. This omission can lead to issues with name uniqueness and potential conflicts if the same name is used across different profiles.

Vulnerability Details

The current implementation of the `changeName` function does not check if a new name is already reserved or if the current name should be de-reserved when changing to a new one. This can result in:

- Multiple profiles having the same name, leading to confusion and potential misuse.
- Inability to enforce unique names across profiles, which might be a requirement for the application.

Recommendation

- Integrate a name reservation mechanism to ensure that names are unique and managed appropriately. This involves checking if a name is already reserved before assigning it and de-reserving the old name when a change occurs.

```
  function changeName(uint256 id, string calldata name) public returns (uint256) {
    Profile memory item = getItem(id);
    address oldOwner = getItemOwner(item);
    _assertOrAssignWallet(item);
    _assertStatus(item, 0);
    require(validateName(name), "Not a valid new name");
     require(sha256(bytes(name)) != sha256(bytes(item.name)), "New name is same as the current one");
    require(!isNameReserved(name), "Name already reserved");

    // If already named, dereserve old name
    if (bytes(item.name).length > 0) {
        toggleReserveName(item.name, false);
    }
    toggleReserveName(name, true);

    item.name = name;
    item.status = 0;
    items[id] = item;
```

```
    address newOwner = getItemOwner(item);

    if (newOwner != oldOwner) {

        _transfer(oldOwner, newOwner, id);

    }

    _setItemIdByWallet(item, id);

    emit ItemUpdated(id, item.status);

    return id;

}


function toggleReserveName(string memory str, bool isReserve) internal {

    _nameReserved[toLower(str)] = isReserve;

}
```

Status

Resolved

[M-04] ProfileWorkflow#_assertOrAssignWallet - Incorrect State Mutability

Description

The `_assertOrAssignWallet` function in the `ProfileWorkflow` contract is incorrectly marked with the `view` modifier, indicating that it should not modify any state variables. However, the function attempts to modify the `wallet` field of the `Profile` struct, which is not permissible under the `view` constraint.

Vulnerability Details

State Mutability Violation: The function is intended to potentially update the `wallet` field of a `Profile` struct. Since the `Profile` is passed as a `memory` parameter, any modifications do not affect the actual stored data. Furthermore, marking the function

as `view` is misleading because it suggests that no state changes will occur, which is not the case if the function's logic were correctly implemented to update state.

Recommendation

To resolve this issue, remove the `view` modifier from the function and change the parameter type from `memory` to `storage` to allow the function to correctly update the state of the `Profile` struct.

```
function _assertOrAssignWallet(Profile storage item) private {

    address wallet = item.wallet;

    require(wallet == address(0) || _msgSender() == wallet, "Invalid Wallet");

    if (wallet == address(0)) {

        item.wallet = _msgSender();

    }

}
```

Status

Acknowledged

[M-05] ProfileWorkflow#_setItemIdByWallet - Overwriting Existing Mappings

Description

The `_setItemIdByWallet` function in the `ProfileWorkflow` contract allows overwriting an existing mapping entry if the new `id` matches `item.id`. This behavior may not always be desired, especially if `item.id` and `id` are different, leading to potential data inconsistency and unintended overwriting of mappings.

Vulnerability Details

Overwriting Existing Mappings: The function checks if an existing mapping entry for a wallet address can be overwritten based on whether the existing item ID matches the current item's ID. If the IDs match, it allows the overwrite, but this logic does not

account for scenarios where `item.id` and `id` are different. This can lead to overwriting existing mappings unintentionally, causing data inconsistency.

Recommendation

To prevent unintended overwriting of existing mappings, modify the function to ensure that it only updates the mapping when it is safe to do so. This may involve additional checks to ensure the integrity and consistency of the mapping entries.

```
function _setItemIdByWallet(Profile storage item, uint id) private {

    if (item.wallet == address(0)) return;

    uint existingItemByWallet = itemsByWallet[item.wallet];

    require(

        existingItemByWallet == 0 || existingItemByWallet == id,

        "Cannot set Wallet. Another item already exists with same value."

    );

    itemsByWallet[item.wallet] = id;

}
```

Status

Resolved

Low

[L-01] ProfileWorkflow#initialize - Hardcoded ERC721 Name and Symbol

Description

The `ProfileWorkflow` contract has hardcoded values for the ERC721 token name and symbol within the `initialize` function. This approach limits the flexibility of the contract, as it requires recompilation and redeployment to change these values, which may not be feasible or efficient for different use cases or environments.

Vulnerability Detail

Hardcoding the ERC721 token name and symbol can lead to several issues:

- Lack of Flexibility: The contract cannot be easily adapted to different projects or versions that may require distinct names and symbols.
- Increased Maintenance: Any change to the token name or symbol necessitates modifying the source code, recompiling, and redeploying the contract, which can be time-consuming and error-prone.
- Potential for Errors: Hardcoded values increase the risk of deploying the contract with incorrect or outdated information, leading to inconsistencies across different environments.

Recommendation

To enhance flexibility and reduce maintenance overhead, modify the `initialize` function to accept the ERC721 token name and symbol as parameters. This allows for dynamic assignment during contract deployment, catering to different use cases without altering the contract code.

```
function initialize(address _newOwner, string memory tokenName, string memory tokenSymbol) public initializer {

    __Ownable_init(_newOwner);

    __NonTransferrableERC721_init();

    __ERC721_init(tokenName, tokenSymbol);

    __WorkflowBase_init();

}
```

Status

Resolved

[L-02] ProfileWorkflow#initialize - Missing Access Control

Description

The initialize function in the ProfileWorkflow contract lacks explicit access control mechanisms to ensure that it can only be called once at deployment or by the deployer contract. This absence of access control could lead to unauthorized re-initialization of the contract, potentially altering its state and compromising its integrity.

Vulnerability Detail

Without proper access control, the initialize function is vulnerable to being called multiple times by arbitrary addresses. This can result in:

- State Manipulation: Unauthorized users could reinitialize the contract, resetting its state and potentially disrupting its intended functionality.
- Security Risks: Re-initialization could be exploited to change ownership or reset critical parameters, leading to loss of control over the contract.
- Integrity Issues: The contract's integrity could be compromised if its initialization logic is executed more than once.

Recommendation

To mitigate this vulnerability, implement access control measures to restrict the `initialize` function to be callable only once and only by authorized entities. This can be achieved by using the `onlyInitializing` modifier provided by OpenZeppelin's `Initializable` contract or by implementing a custom access control mechanism.

```
function initialize(address _newOwner) public initializer onlyInitializing {

    __Ownable_init(_newOwner);

    __NonTransferrableERC721_init();

    __ERC721_init("Profile - PROFILE test profile v1", "PROFILE");

    __WorkflowBase_init();

}
```

Status

Resolved

[L-03] ProfileWorkflow#initialize - Vulnerable to Frontrunning

Description

The initializer function in the `ProfileWorkflow` contract is vulnerable to frontrunning attacks. This vulnerability allows an attacker to potentially set their own values, take ownership of the contract, or force a redeployment, compromising the security and integrity of the contract.

Vulnerability Detail

- Frontrunning Vulnerability: The initializer function can be called by any address, allowing an attacker to execute the function before the intended deployer. This could result in the attacker setting critical parameters, such as ownership, to their own address.
- Ownership Risks: If an attacker frontruns the initializer, they could take ownership of the contract, leading to unauthorized control over its functions and data.
- Deployment Issues: In the best-case scenario, a successful attack would force a redeployment of the contract, incurring additional costs and delays.

Recommendation

```
function initialize(address _newOwner) public initializer {

    __Ownable_init(_newOwner);

    __NonTransferrableERC721_init();

    __ERC721_init("Profile - PROFILE test profile v1", "PROFILE");

    __WorkflowBase_init();

}
```

To mitigate the risk of frontrunning, implement access control measures to restrict who can call the initializer function. This can be achieved by ensuring that the initializer can only be called once and only by the intended deployer or a trusted entity.

```
function initialize(address _newOwner) public initializer {

    require(msg.sender == trustedDeployer, "Unauthorized initializer call");

    __Ownable_init(_newOwner);

    __NonTransferrableERC721_init();

    __ERC721_init("Profile - PROFILE test profile v1", "PROFILE");

    __WorkflowBase_init();

}
```

Note

It's not relevant in context of TransparentUpgradeableProxy, Given the contract when deployed has no state (no constructor logic and no state as per OpenZeppelin instructions), there would be no "trustedDeployer" set yet. Furthermore, upon deployment, the Initialize method will be run by TransparentUpgradeableProxy (more precisely ERC1967Utils.upgradeToAndCall()) which sets the implementation address and calls Initialize(). This all happens within the same block transaction. This is a general warning in OZ contract ("TIP: To avoid leaving the proxy in an uninitialized state, the initializer function should be called as early as possible", refer Initializable by OpenZeppelin

Status

Acknowledged

[L-04] ProfileWorkflow#registerProfile - Potential Issue with `_mint` Usage in ERC721 Token Minting

Description

The `registerProfile` function in the `ProfileWorkflow` contract uses the `_mint` function to mint ERC721 tokens. This approach can lead to issues if tokens are minted to

addresses that do not support ERC721 tokens. To ensure compatibility and avoid potential token loss, it is recommended to use `_safeMint` instead of `_mint`.

Vulnerability Detail

- Compatibility Risk: Using `_mint` does not check whether the recipient address is capable of handling ERC721 tokens. If the recipient is a contract that does not implement the ERC721Receiver interface, the tokens could be permanently locked or lost.
- Lack of Safety Checks: The `_mint` function lacks built-in safety checks to verify that the recipient can handle ERC721 tokens, which `_safeMint` provides by calling `onERC721Received` on the recipient if it is a contract.

Recommendation

To mitigate the risk of minting tokens to incompatible addresses, replace `_mint` with `_safeMint` in the `registerProfile` function. This change ensures that tokens are only minted to addresses that can handle them properly.

```
function registerProfile(string calldata name) public returns (uint256) {

    uint256 id = _getNextId();

    Profile memory item;

    item.id = id;

    _assertOrAssignWallet(item);

    _setItemIdByWallet(item, 0);

    item.name = name;

    item.status = 0;

    items[id] = item;

    address newOwner = getItemOwner(item);

    _safeMint(newOwner, id); // Use _safeMint instead

    _setItemIdByWallet(item, id);

    emit ItemUpdated(id, item.status);
```

```
    return id;

}
```

Status

Resolved

Centralisation

The RDDTOR project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlocks analysis, the RDDTOR project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au


#Hashlock.